

my_cp — Design Document

GitHub repository: https://github.com/Roybo1508/System_Programming_ex1

1. Purpose and Requirements Overview

`my_cp` is a command line program that copies one file to another using low level system calls instead of the `stdio` library functions. The size of the chunk that is copied in each iteration (the granularity) is given by the user, and the program measures how long the copy itself took.

The program is run like this:

```
./my_cp <source_file> <destination_file> <granularity_bytes>
```

How the program meets the requirements of the assignment:

- **CLI interface with 3 arguments** — the program checks that `argc == 4` (the program name plus the three arguments). If the number of arguments is different it prints a usage message to `stderr` and exits with `EXIT_FAILURE`. The third argument is converted from a string to an int with `atoi()` and must be greater than 0.
- **Dynamic buffer allocation** — there is no fixed size array in the program. The buffer is allocated with `malloc(granularity)` after the granularity was validated, the returned pointer is checked against `NULL`, and the buffer is released with `free()` before every exit from the program, both on success and on failure.
- **I/O with system calls** — the files are opened with `open()`, the data is copied with `read()` and `write()`, and the files are closed with `close()`. No `fopen`, `fread` or `fwrite` are used for the copying itself. `printf` and `fprintf` are used only for printing messages to the screen.
- **Internal timing** — the program measures the time itself with `clock_gettime(CLOCK_MONOTONIC, ...)`, and prints the result in milliseconds with three digits after the decimal point.

2. System Calls Analysis

`open()`

`open()` asks the operating system to open a file and returns a file descriptor, which is a small integer that identifies the open file. Every later call to `read()`, `write()` or `close()` uses this number to say which file it works on. If the call fails it returns -1.

The program calls `open()` twice, with different flags, because the two files are used in different ways:

Source file — `open(argv[1], O_RDONLY)`

- `O_RDONLY` — open the file for reading only. This is all that is needed because the program only reads from the source, and it also means that the program will not change the source file by mistake.

Because `O_CREAT` is not used here, the call fails if the source file does not exist, which is the behaviour we want — there is nothing to copy from.

Destination file — `open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644)`

The three flags are combined with the bitwise OR operator `|` :

- `O_WRONLY` — open the file for writing only, since the program never reads the destination.
- `O_CREAT` — if the destination file does not exist, create it. Without this flag copying to a new file name would fail.
- `O_TRUNC` — if the destination file already exists, empty it (set its size to 0) before writing. Without this flag, copying a small file over a bigger existing file would leave the old bytes at the end of the file and the result would be wrong.

The last argument `0644` is the permissions of the file, and it is only used when the file is actually created by `O_CREAT` . In octal it means read and write for the owner of the file, and read only for the group and for everyone else.

read()

`read(src_fd, buffer, granularity)` copies up to `granularity` bytes from the source file into the buffer, and returns how many bytes it actually read. The return value is important because it is not always the number that was asked for:

- A positive number — this is how many bytes were really read, and only this many bytes should be written to the destination.
- 0 — end of file. There is nothing left to copy, so the loop stops.
- -1 — the call failed.

write()

`write(dst_fd, buffer + total_written, bytes_read - total_written)` writes bytes from the buffer into the destination file and returns how many bytes it actually wrote. Like `read()` , this number can be smaller than the number that was asked for, so the program cannot assume that one call to `write()` wrote everything. A return value of -1 means the call failed.

close()

`close()` releases the file descriptor back to the operating system. A process can only have a limited number of open files, so closing them when they are not needed anymore is part of cleaning up correctly. It returns -1 if it failed. The program closes both file descriptors on the successful path and also on every error path.

clock_gettime()

`clock_gettime(CLOCK_MONOTONIC, &t)` fills a `struct timespec` with the current time, in two fields: `tv_sec` for whole seconds and `tv_nsec` for nanoseconds. It returns -1 if it failed.

`CLOCK_MONOTONIC` is a clock that only moves forward and is not affected by changes to the system clock. This is why it fits measuring how long an operation took: if the system clock is changed while the program runs, the measurement is still correct.

3. Flow Logic

Step 1 — Argument check. The program checks `argc != 4`. If so, it prints a usage message to `stderr` and returns `EXIT_FAILURE`. This check has to be first, because every step after it reads `argv[1]`, `argv[2]` or `argv[3]`.

Step 2 — Granularity validation. `atoi(argv[3])` converts the third argument to an int. `atoi` returns 0 for text that is not a number, so a single check of `granularity <= 0` rejects both invalid text and values that make no sense such as 0 or a negative number. On failure the program prints an error to `stderr` and returns `EXIT_FAILURE`.

Step 3 — Buffer allocation. `malloc(granularity)` allocates the buffer on the heap. If it returns `NULL` the program prints an error and exits. Nothing is open yet at this point, so there is nothing to clean up.

Step 4 — Opening the source file. `open()` with `O_RDONLY`. If the return value is -1 the program calls `perror()`, frees the buffer, and returns `EXIT_FAILURE`.

Step 5 — Opening the destination file. `open()` with `O_WRONLY | O_CREAT | O_TRUNC` and mode `0644`. If it fails, the source file is already open at this point, so the cleanup here is bigger: `perror()`, `close(src_fd)`, `free(buffer)`, and then return `EXIT_FAILURE`.

Step 6 — Starting the timer. `clock_gettime(CLOCK_MONOTONIC, &start_time)` is called here on purpose: after both files were opened successfully and before the copy loop starts, so that the time of opening the files is not counted in the measurement.

Step 7 — The main copy loop. The loop runs until the end of the source file:

1. `read()` reads up to `granularity` bytes into the buffer.
2. If `read()` returned -1 with `errno == EINTR`, the loop continues and does the same read again. Any other -1 is a real error, so the program cleans up and exits.
3. If `read()` returned 0, this is the end of the file and the loop breaks.
4. Otherwise there are `bytes_read` bytes in the buffer that have to be written. An inner loop keeps calling `write()` until `total_written` reaches `bytes_read`. Each call writes from `buffer + total_written`, which is the first byte that was not written yet, and asks for `bytes_read - total_written` bytes, which is how much is left. After each successful call `total_written` grows by the number of bytes that were actually written.

5. When the inner loop finishes, the whole chunk is in the destination file and the outer loop goes back to the next `read()`.

Step 8 — Stopping the timer. `clock_gettime(CLOCK_MONOTONIC, &end_time)` is called right after the loop finished and before the files are closed, so that the time of closing the files is not counted either.

Step 9 — Closing the files and freeing the buffer. Both file descriptors are closed with `close()`, and the buffer is released with `free()`. The return value of each `close()` is checked, and if it failed the program reports it with `perror()`, but it does not exit with a failure code, because at this stage the copy already finished successfully.

Step 10 — Calculating and printing the time. The seconds and the nanoseconds of the two measurements are subtracted separately. If the result of the nanoseconds is negative, the program borrows one second: it subtracts 1 from the seconds and adds 1,000,000,000 to the nanoseconds. This is needed because the time is kept in two separate fields, so it is possible that the nanoseconds field of the end time is smaller than the one of the start time even though the end time is later. After that, the seconds are multiplied by 1000 and the nanoseconds are divided by 1,000,000, so both are in milliseconds, and they are added together and printed with `%.3f`. The program returns `EXIT_SUCCESS`.

4. Error Handling

Checking return values. Every system call in the program is checked. `open()`, `read()`, `write()`, `close()` and `clock_gettime()` all return -1 when they fail, and `malloc()` returns `NULL`.

`perror()`. When a system call fails, the operating system puts the reason of the failure in the global variable `errno`. `perror("open source file")` prints the string that was given to it, and after it the explanation of the error in words, for example “No such file or directory”. This is more useful than a general message, because it says exactly what went wrong. `perror` is used for the failures of the system calls, while `fprintf(stderr, ...)` is used for the errors that are not system call errors, such as the wrong number of arguments or an invalid granularity.

`EINTR` retry. A system call that is blocked and waiting can be interrupted by a signal that arrives to the process. When this happens the call returns -1 and sets `errno` to `EINTR`. This is not a real error: no data was read or written and nothing was lost. Because of that, both in `read()` and in `write()` the program checks whether `errno == EINTR`, and if so it does `continue` and tries the same operation again instead of stopping the copy. Only if `errno` is something else the program treats it as a real failure.

Partial writes. `write()` is allowed to write fewer bytes than it was asked for. If the program ignored this and moved on to the next chunk, some bytes would be lost and the copied file would be wrong. This is why the inner loop keeps writing until all the bytes that were read were also written, and each call continues from the place in the buffer where the previous one stopped.

Cleanup on every path. On every error path that happens after resources were taken, the program releases everything it took before it exits: it closes the file descriptors that are already open and frees the buffer. This way the program does not leak memory or leave open file descriptors, no matter where it failed.